

Network-on-Chip (NoC) Design for Transformers

Rupak Antani
2022102045
rupak.antani@students.iiit.ac.in

Abstract—Transformer-based architectures have become the foundation of modern Artificial Intelligence systems and Large Language Models (LLMs). These models involve massive matrix multiplications, attention computations and nonlinear operations that demand extremely high computational throughput and communication bandwidth. Traditional accelerator architectures suffer from scalability limitations, memory bottlenecks and inefficient data movement when handling transformer workloads.

Network-on-Chip (NoC) architectures provide scalable and high-bandwidth communication infrastructure capable of efficiently interconnecting Processing Elements (PEs), memory blocks, and specialized compute units. This report presents a detailed study of NoC architectures for transformer accelerators with special focus on the NOCTUA architecture, a high-efficiency reconfigurable NoC-based transformer accelerator. This report discusses the communication requirements of transformer operations, challenges in existing NoC architectures, details of the NOCTUA design, hardware-friendly nonlinear implementations, implementation methodology, obtained results and future research directions.

I. INTRODUCTION

Artificial Intelligence (AI) has undergone tremendous advancement over the past decade, particularly due to the emergence of transformer-based architectures. Transformers have become the core building block of modern AI systems such as GPT, BERT, Llama, Vision Transformers (ViTs), etc. Unlike traditional neural networks, transformers rely heavily on self-attention mechanisms that allow the model to understand long-range dependencies between input tokens. Although transformers provide state-of-the-art accuracy, they require extremely high computational and communication resources. Transformer models involve large matrix multiplication, frequent memory accesses, massive tensor movement and parallel execution across many compute units.

As model sizes continue to grow into billions of parameters, the communication overhead inside the accelerator becomes a critical bottleneck. In many cases, the energy consumed for moving data is significantly larger than the energy consumed for computation itself. Traditional communication architectures such as shared buses are insufficient for handling high bandwidth communication, massive parallelism, scalable interconnections and dynamic transformer workloads.

To overcome these limitations, modern transformer accelerators increasingly use Network-on-Chip (NoC) architectures. A Network-on-Chip (NoC) is a packet-switched communica-

tion framework that interconnects Processing Elements (PEs), memory blocks, controllers and specialized compute units. The NoC acts as the communication backbone of the accelerator and directly impacts throughput, latency, power consumption, scalability and PE utilization. This report studies the importance of NoC architectures in transformer accelerators and presents a detailed discussion of the NOCTUA architecture.

II. ROLE OF NOC IN TRANSFORMER ACCELERATORS

Transformer accelerators contain multiple hardware modules operating simultaneously. These modules continuously exchange tensors, partial sums, weights and activation data. The NoC is responsible for:

- Efficient communication between PEs
- Data broadcasting
- Reduction operations
- Synchronization
- Memory access coordination

A. Importance of Communication in Transformers

In transformers, the same tensor is often reused multiple times. For example, weight matrices are shared across multiple PEs, attention scores require reductions and partial outputs must be accumulated. This creates massive communication traffic inside the chip. Communication overhead leads to:

- Increased latency
- Higher energy consumption
- Reduced throughput
- PE underutilization

Therefore, an efficient NoC design becomes essential.

B. Transformer Operations and NoC Requirements

Different transformer operations require different communication patterns as shown in table I.

TABLE I
TRANSFORMER OPERATIONS AND HARDWARE REQUIREMENTS

Operation	Computation	Preferred NoC
Q, K, V Projection	GEMM	2D Mesh
QK ^T Attention	GEMM	2D Mesh
Softmax	Reduction	Tree / 1D
AV Multiplication	GEMM	2D Mesh
FFN	GEMM	1D + 2D Mesh

1) *Q, K and V Projection*

The Query, Key and Value projection layers are computed as:

$$Q = XW_Q$$

$$K = XW_K$$

$$V = XW_V$$

These are large matrix multiplication operations require parallel communication, high bandwidth and efficient data reuse. Therefore, 2D mesh NoC architectures are highly efficient for such operations.

2) *Attention Score Computation*

The attention score matrix is computed as:

$$Attention = QK^T$$

This operation involves large tensor movement, PE-to-PE communication and partial sum accumulation. Thus, efficient NoC routing becomes critical.

3) *Softmax Operation*

The Softmax operation is defined as:

$$Softmax(x_i) = \frac{e^{x_i}}{\sum_j e^{x_j}}$$

Softmax requires reduction operations, broadcast operations and global synchronization. Tree-based or 1D NoC topologies are more efficient for these operations.

4) *Feed Forward Network (FFN)*

The FFN layer contains matrix multiplication, GELU activation and layer normalization. These operations require mixed communication patterns.

III. PROBLEMS IN CURRENT NOC ARCHITECTURES

Although NoCs provide scalable communication, existing architectures still face several major challenges that reduce efficiency, scalability, and flexibility in transformer accelerators.

1) **Fixed Dataflow Architectures**

Many transformer accelerators use fixed dataflow mechanisms such as Weight stationary, Output stationary and Input stationary. Although these approaches are optimized for specific workloads, they are not flexible enough to efficiently support the varying computation patterns present in transformer models. Different transformer layers often require different data movement strategies, and fixed dataflows cannot dynamically adapt to these requirements. As a result, fixed dataflow architectures lead to:

- Poor architectural flexibility
- Processing Element (PE) underutilization
- Inefficient execution for varying matrix dimensions

- Increased execution latency

2) **Bus-based Communication Bottlenecks**

Traditional communication architectures often rely on shared buses for data transfer between compute units and memory blocks. However, shared buses become inefficient when the number of Processing Elements increases. Bus-based communication suffers from:

- Limited bandwidth
- High communication contention
- Poor scalability
- Increased packet congestion

As transformer accelerators scale to hundreds or thousands of Processing Elements, the shared communication medium becomes a major bottleneck, significantly reducing overall throughput.

3) **Memory Bottleneck**

Transformer workloads require continuous movement of large tensors between memory and compute units. Frequent access is required for weight matrices, activation tensors, attention score matrices and intermediate outputs. This creates extremely high memory traffic inside the accelerator. In many cases, the energy consumed for memory access is larger than the energy consumed for arithmetic operations. Memory bottlenecks result in:

- Increased latency
- High power consumption
- Reduced throughput
- Bandwidth limitations

4) **Poor Processing Element Utilization**

Transformer models involve dynamically changing matrix sizes and irregular workloads. Fixed PE arrays are unable to efficiently adapt to these changing requirements. Consequently, some PEs remain idle, hardware resources are wasted, parallelism reduces significantly and the overall accelerator efficiency decreases. Poor PE utilization directly affects the throughput, energy efficiency and area efficiency.

5) **Fixed NoC Topology Limitation**

Most conventional NoC architectures are designed using a single fixed topology such as 2D Mesh, 1D Ring or Tree topology. However, transformer workloads consist of multiple operations with very different communication requirements. For example:

- Matrix multiplications such as QKV projection and QK^T attention work efficiently on 2D mesh architectures due to their parallel communication capabilities.
- Reduction-based operations such as Softmax and Layer Normalization are better suited for tree-based or 1D communication structures.

- Feed Forward Networks (FFNs) may require hybrid communication patterns involving both 1D and 2D communication.

Therefore, a single fixed NoC topology cannot efficiently support all transformer operations.

One possible solution is to use multiple NoCs with different topologies inside the same accelerator. However, this approach is highly impractical because it leads to:

- Large hardware overhead
- Increased chip area
- Higher power consumption
- Complex routing and control logic
- Increased design complexity

As transformer models continue to evolve, the inability of fixed NoC architectures to dynamically adapt to different communication patterns becomes a major limitation. This creates the need for reconfigurable NoC architectures capable of dynamically changing topology based on the transformer operation being executed.

IV. NOCTUA

NOCTUA (Network-on-Chip Transformer Universal Accelerator) is a high-efficiency reconfigurable NoC-based transformer accelerator designed to address the limitations of conventional transformer accelerator architectures. The work was proposed by Kun-Chih (Jimmy) Chen, Pin-Ching Shen, and Bo-Chun Chen from the Institute of Electronics, National Yang Ming Chiao Tung University, Taiwan. The paper titled “*NOCTUA: A High-Efficiency Reconfigurable NoC-based Transformer Universal Accelerator*” was published in the 2025 Asia Pacific Signal and Information Processing Association Annual Summit and Conference (APSIPA ASC 2025).

The primary motivation behind NOCTUA is that transformer workloads involve highly diverse communication and computation patterns. Conventional accelerators generally use fixed Processing Element (PE) arrays and fixed NoC topologies, which are unable to efficiently adapt to:

- Different transformer layers
- Varying matrix dimensions
- Reduction-based operations
- Dynamic workloads

As a result, traditional architectures often suffer from poor PE utilization, high communication overhead, memory bottleneck and reduced scalability.

To solve these problems, NOCTUA introduces a dynamically reconfigurable NoC architecture capable of changing communication topologies and dataflow mechanisms depending on the transformer operation being executed. The proposed architecture is designed to support multiple transformer models including BERT, GPT-2, Attention-based NLP models, etc. Figure 1 shows the difference between the conventional PE accelerator and the proposed NoC-based accelerator in the

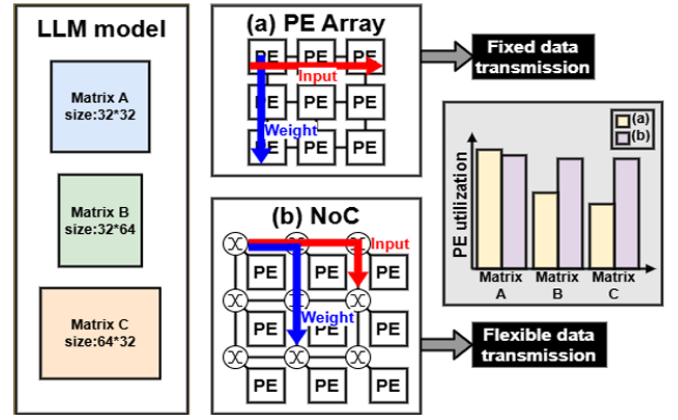


Fig. 1. (a) The conventional PE array accelerator and (b) the proposed NoC-based accelerator apply with LLM models, and the proposed method outperforms at PE utilization.

paper. The paper highlights that NOCTUA achieves high operational efficiency while maintaining flexibility and scalability for future transformer workloads.

A. Objectives of NOCTUA

The main objectives of the NOCTUA architecture are:

- Dynamic NoC reconfiguration based on transformer operation requirements
- Flexible dataflow mapping to efficiently support varying matrix dimensions
- Reduction of communication overhead and memory traffic
- Improved Processing Element (PE) utilization
- Efficient implementation of nonlinear transformer operations such as Softmax, GELU and Layer Normalization.
- Scalable architecture capable of supporting future transformer models

B. Key Features of NOCTUA

The NOCTUA architecture introduces several important architectural innovations:

- **Reconfigurable NoC Architecture** – The NoC topology can dynamically switch between 2D Mesh, Tree topology and 1D communication structures depending on the transformer operation being executed.
- **Heterogeneous Processing Elements** – Different PEs contain different hardware resources. Some PEs contain specialized nonlinear units while others are optimized for matrix multiplication.
- **Efficient Multicast Communication** – Shared tensors such as weights and activations can be distributed to multiple PEs simultaneously, reducing communication overhead and memory accesses.
- **Dynamic Routing and Dataflow Adaptation** – Routing paths and dataflow mechanisms are dynamically modified based on workload characteristics.

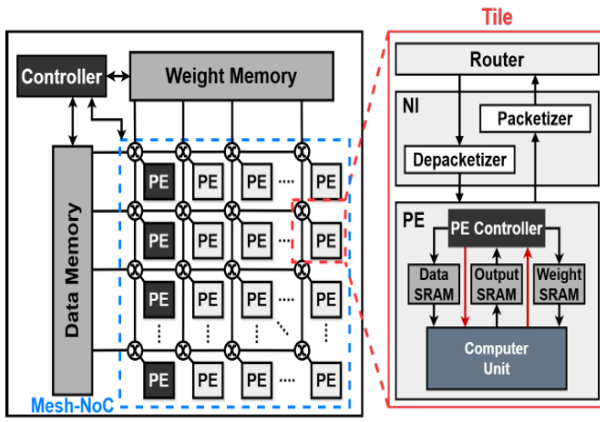


Fig. 2. Overall NOCTUA Architecture

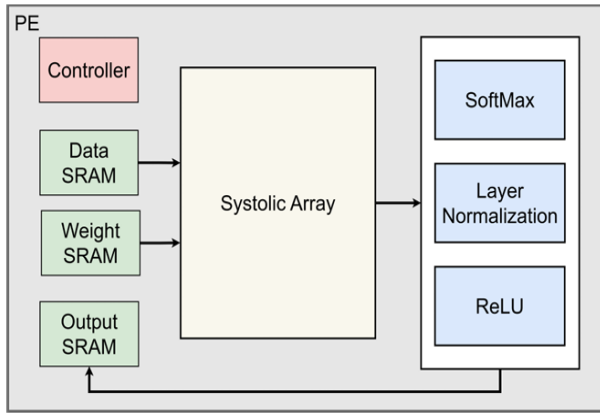


Fig. 3. PE Architecture

- **Hardware-Friendly Nonlinear Functions** – NOCTUA implements optimized hardware approximations for Softmax, GELU activation and Layer Normalization to reduce hardware complexity and improve execution efficiency.

V. NOCTUA ARCHITECTURE

A. Overall Architecture

Figure 2 shows the complete NOCTUA architecture. It consists of:

- **Central controller** – It manages dataflow scheduling, topology selection, memory allocation and PE coordination.
- **2D mesh NoC** – NOCTUA uses a 2D mesh NoC. This gives it several advantages such as scalable design, parallel communication, simple routing and lower congestion.
- **Processing Elements (PEs)** – Each PE contains PE controller, weight SRAM, data SRAM, output SRAM and compute unit. Figure 3 shows the internal architecture of a PE.
- **On-chip memory**
- **Specialized nonlinear units**

The architecture dynamically reconfigures based on transformer workload requirements. The leftmost column PEs additionally contain Softmax units, Layer normalization units and GELU/ReLU units. This reduces area overhead while supporting transformer-specific nonlinear operations. Also, NOCTUA dynamically changes routing paths, communication patterns and data distribution methods depending on transformer stage requirements.

B. Hardware-Friendly Nonlinear Implementations

1) GELU Approximation

The standard GELU activation is computationally expensive. NOCTUA uses LUT-based approximation and Hybrid GELU-ReLU implementation. The GELU function is given as:

$$GELU(x) = x\Phi(x)$$

where $\Phi(x)$ is the Gaussian cumulative distribution function. NOCTUA uses LUT approximation for $-4 \leq x \leq 2$. Outside this range, ReLU is used. This gives several advantages such as reduced LUT size, lower latency and reduced hardware complexity.

2) Integer Softmax

The standard Softmax operation requires exponentiation and division which are expensive in hardware. NOCTUA implements integer arithmetic, dynamic scaling and bit-shift operations. This significantly reduces hardware cost.

3) Layer Normalization

Layer normalization is defined as:

$$y = \frac{x - \mu}{\sqrt{\sigma^2 + \delta}} \cdot \gamma + \beta$$

where μ is the mean and σ^2 is the variance. NOCTUA uses piecewise Linear approximation and Newton-Raphson inverse square root. The Newton-Raphson iteration is:

$$s_{k+1} = 0.5s_k(3 - Vs_k^2)$$

This avoids expensive square root and division hardware.

VI. NOCTUA RESULTS

NOCTUA successfully demonstrated all major transformer stages. NOCTUA achieved:

- Throughput: 32.77 TOPS
- Energy Efficiency: 1.05 TOPS/W
- Area Efficiency: 0.71 TOPS/mm²

Also, NOCTUA was evaluated using some existing transformer accelerators. All the results and comparisons of NOCTUA are summarized in table II.

TABLE II
COMPARISON WITH EXISTING TRANSFORMER ACCELERATORS

Feature	ITA	RAWAtten	DTQAtten	NOCTUA
Technology	22nm	40nm	40nm	40nm
Frequency	500MHz	1GHz	1GHz	1GHz
MAC Units	1024	1024	3168	16384
Throughput	1.02 TOPS	0.77 TOPS	0.95 TOPS	32.77 TOPS
Energy Efficiency	8.46 TOPS/W	1.17 TOPS/W	1.3 TOPS/W	1.05 TOPS/W
Area Efficiency	2.52 TOPS/mm ²	0.47 TOPS/mm ²	0.68 TOPS/mm ²	0.71 TOPS/mm ²

VII. IMPLEMENTATION

The proposed NOCTUA architecture was implemented and analyzed using a simulation-based approach to evaluate its ability to efficiently execute transformer workloads while dynamically adapting the Network-on-Chip topology and dataflow mechanisms. The implementation mainly focuses on:

- Dynamic NoC topology reconfiguration
- Flexible dataflow selection
- Efficient transformer operation execution
- Improved Processing Element utilization

The complete implementation can be found on SARCS-IIITH's Network-on-Chip-NoC repository .

A. Implementation Environment

The NOCTUA architecture was implemented using the gem5 simulator. The simulator was used to model Processing Element communication, NoC routing behavior, dynamic topology reconfiguration and transformer workload execution.

B. Overall Architecture

The implemented NOCTUA architecture consists of two major blocks:

- Central Controller
- Systolic Array Processing Elements

The central controller manages the entire accelerator operation, while the systolic array performs transformer computations such as matrix multiplication, attention computation, softmax and layer normalization. The architecture dynamically reconfigures communication patterns depending on the transformer stage being executed.

C. Central Controller

The Central Controller acts as the brain of the NOCTUA architecture and is responsible for controlling the complete execution flow. It mainly contains:

- Topology Manager
- Dataflow Controller

The controller continuously monitors workload requirements and dynamically configures the NoC topology, dataflow strategy and PE communication paths. This allows the accelerator to efficiently support multiple transformer operations using the same hardware resources.

1) Topology Manager

The Topology Manager is responsible for dynamically reconfiguring the NoC communication topology based on the operation currently being executed. The manager can switch between:

- 2D Mesh topology
- Tree topology
- 1D NoC / Ring topology

NOCTUA's ability to dynamically reconfigure topology eliminates the need for multiple separate NoCs inside the accelerator.

2) Dataflow Controller

The Dataflow Controller dynamically selects the most suitable dataflow mechanism depending on the transformer workload. The supported dataflow strategies include:

- Input stationary
- Weight stationary
- Output stationary

The controller dynamically chooses the optimal dataflow based on matrix dimensions, communication requirements, memory access patterns and transformer layer type. This significantly improves PE utilization, throughput and energy efficiency.

D. Systolic Array Processing Elements

The NOCTUA architecture contains Processing Elements organized in the form of systolic arrays. The systolic array structure enables parallel matrix multiplication, efficient tensor movement, localized communication and high throughput computation. The PE structure is divided into:

- Left Column PEs
- Other PEs

1) Left Column Processing Elements

The Left Column PEs are specialized Processing Elements that contain additional nonlinear hardware units. These PEs include softmax units, layer normalization units and the normal systolic array hardware. These specialized units are mainly required during 1D NoC operations. By placing these units only in selected PEs, NOCTUA reduces area overhead, hardware redundancy and power consumption.

2) Other Processing Elements

The remaining Processing Elements mainly contain the standard systolic array hardware. These PEs are optimized

primarily for matrix multiplication, attention computation and tensor propagation. Since most transformer operations are dominated by matrix multiplication, these PEs form the primary computation backbone of the accelerator.

E. Dynamic Reconfiguration Flow

During execution, the NOCTUA architecture dynamically performs:

- 1) Analysis of transformer operation type
- 2) Selection of optimal NoC topology
- 3) Selection of suitable dataflow strategy
- 4) Routing configuration
- 5) Processing Element allocation

This dynamic adaptation allows NOCTUA to efficiently execute all transformer stages using a single reconfigurable architecture.

F. Implementation Results

The complete NOCTUA architecture was successfully implemented in gem5 simulator and the transformer operations were demonstrated stage-by-stage by taking sample input values. This confirmed that NOCTUA is efficiently able to reconfigure and adapt based on the needs of each stage of a transformer.

Figure 4 shows the demonstration of the stage 1 of the transformer, which is the attention score calculation. For this stage, the NOCTUA NoC was first configured to 2D Mesh topology. After this, sample Q and K^T matrices were given as inputs to NOCTUA and the correct attention score matrix was obtained as output.

STAGE 1: Q-K ^T - Attention Score Calculation Topology: 2D MESH												
NOCTUA reconfiguring NoC to 2D MESH NoC topology												
INPUT Q (Query):												
1.0000	2.0000	3.0000	4.0000									
5.0000	6.0000	7.0000	8.0000									
9.0000	10.0000	11.0000	12.0000									
13.0000	14.0000	15.0000	16.0000									
INPUT K ^T :												
1.0000	0.0000	0.0000	0.0000									
0.0000	1.0000	0.0000	0.0000									
0.0000	0.0000	1.0000	0.0000									
0.0000	0.0000	0.0000	1.0000									
NOCTUA Systolic Array computing Q-K ^T using 2D MESH NoC topology:												
OUTPUT Attention Scores = Q-K ^T :												
1.0000	2.0000	3.0000	4.0000									
5.0000	6.0000	7.0000	8.0000									
9.0000	10.0000	11.0000	12.0000									
13.0000	14.0000	15.0000	16.0000									

Fig. 4. Transformer Stage 1: Attention Score Calculation

Figure 5 shows the demonstration of the stage 2 of the transformer, which is the softmax calculation. Firstly, the NOCTUA NoC was reconfigured to tree NoC topology from 2D Mesh topology for this stage. Next, the attention scores computed in the stage 1 were given as inputs to this stage and the output probabilities were obtained after softmax computation by NOCTUA. The sum of all the obtained probabilities is 1 which confirms that the computed result is correct.

STAGE 2: Softmax - Attention Normalization Topology: TREE												
NOCTUA reconfiguring NoC to TREE NoC topology												
INPUT Attention Scores (first row): [1.0000, 2.0000, 3.0000, 4.0000]												
NOCTUA Softmax Unit computing using TREE NoC topology												
OUTPUT Probabilities: [0.0321, 0.0871, 0.2369, 0.6439]												
Sum = 1.0000												

Fig. 5. Transformer Stage 2: Softmax Computation

Figure 6 shows the demonstration of the stage 3 of the transformer, which is the QKV projection. This stage again requires a 2D Mesh NoC. So, the NOCTUA NoC is reconfigured to 2D Mesh topology from tree topology. Then, the attention score matrix computed in the stage 1 and a sample V matrix are given as inputs to NOCTUA and we get the correct weighted value matrix as the output after performing matrix multiplication using the systolic array.

STAGE 3: xV - Value Projection Topology: 2D MESH												
NOCTUA reconfiguring NoC to 2D MESH NoC topology												
INPUT Attention Scores (from Stage 1):												
1.0000	2.0000	3.0000	4.0000									
5.0000	6.0000	7.0000	8.0000									
9.0000	10.0000	11.0000	12.0000									
13.0000	14.0000	15.0000	16.0000									
INPUT V (Value Matrix):												
1.0000	2.0000	3.0000	4.0000									
5.0000	6.0000	7.0000	8.0000									
9.0000	10.0000	11.0000	12.0000									
13.0000	14.0000	15.0000	16.0000									
NOCTUA Systolic Array computing xV using 2D MESH NoC topology												
OUTPUT Weighted Values = Attention x V :												
90.0000	100.0000	110.0000	120.0000									
202.0000	228.0000	254.0000	280.0000									
314.0000	356.0000	398.0000	440.0000									
426.0000	484.0000	542.0000	600.0000									

Fig. 6. Transformer Stage 3: QKV Projection

Figure 7 shows the demonstration of the stage 4 of the transformer, which is GELU computation in the FFN layer. This stage requires a 1D NoC topology. Hence, the NOCTUA NoC is reconfigured from 2D Mesh to 1D topology. Next, sample input FFN values are given as input and the correct GELU activation values are obtained as output.

STAGE 4: GELU Activation (FFN Layer) Topology: 1D RING												
NOCTUA reconfiguring NoC to 1D NoC topology												
INPUT FFN Values: [-2.0000, -1.5000, -1.0000, -0.5000, 0.0000, 0.5000, 1.0000, 1.5000]												
OUTPUT GELU Activations: [-0.0454, -0.1004, -0.1508, -0.1543, 0.0000, 0.3457, 0.8412, 1.3996]												

Fig. 7. Transformer Stage 4: GELU Computation

Figure 8 shows the demonstration of the stage 5 of the transformer, which is the layer normalization stage. In this stage, the NoC is first reconfigured to 1D topology and then sample input values are given to NOCTUA. After this, we obtain the final normalized values as output.

STAGE 5: Layer Normalization Topology: 1D RING												
NOCTUA reconfiguring NoC to 1D NoC topology												
INPUT Values: [1.0000, 2.0000, 3.0000, 4.0000, 5.0000, 6.0000, 7.0000, 8.0000]												
OUTPUT Normalized Values: [-1.5275, -1.0911, -0.6547, -0.2182, 0.2182, 0.6547, 1.0911, 1.5275]												

Fig. 8. Transformer Stage 5: Layer Normalization Computation

VIII. LIMITATIONS

Despite its advantages, NOCTUA still has few limitations:

- High power consumption
- Complex control logic
- Additional reconfiguration overhead
- Increased area usage

IX. FUTURE WORK

Future research directions include exploring the following things:

- **Benchmarking transformer models** – Perform benchmarking of the implemented NOCTUA architecture on transformer models like BERT, RAWAtten, DTQAtten, etc.
- **Multi-chip NoC architectures** – Explore concepts such as scale-out and figure out how to design NoC for a multi-chip accelerator.
- **Diagonal PE interconnections** – Implement diagonal linking of PEs to reduce the number of clock cycles required in case of tree NoC communication and compare the number of clock cycles required and throughput with the normal 2D Mesh PE architecture.

X. CONCLUSION

Transformer accelerators require highly efficient communication architectures to handle the enormous computational and data movement demands of modern AI systems. Traditional communication infrastructures suffer from scalability limitations and inefficient PE utilization. Network-on-Chip architectures provide scalable and flexible communication solutions capable of efficiently supporting transformer workloads. The NOCTUA architecture demonstrates how reconfigurable NoC designs combined with heterogeneous Processing Elements and adaptive dataflows can significantly improve transformer accelerator performance.

NOCTUA successfully supports multiple transformer operations using dynamically reconfigurable topologies such as 2D mesh, tree, and 1D ring architectures. Its hardware-friendly implementations of Softmax, GELU and Layer Normalization reduce hardware complexity while maintaining high model accuracy. The results show that reconfigurable NoC architectures are highly promising for future transformer accelerators and next-generation AI hardware systems.

XI. REFERENCES

- 1) A. Vaswani et al., “Attention Is All You Need,” NeurIPS, 2017.
- 2) J. Devlin et al., “BERT: Pre-training of Deep Bidirectional Transformers,” NAACL, 2019.
- 3) A. Dosovitskiy et al., “Vision Transformers,” ICLR, 2021.
- 4) Y. Tay et al., “Efficient Transformers: A Survey,” ACM Computing Surveys, 2023.
- 5) T. Brown et al., “Language Models are Few-shot Learners,” NeurIPS, 2020.
- 6) V. Sze et al., “Efficient Processing of Deep Neural Networks,” Proceedings of IEEE, 2017.
- 7) L. Benini and G. De Micheli, “Networks on Chips: A New SoC Paradigm,” Computer, 2002.
- 8) G. Islamoglu et al., “ITA: An Energy-Efficient Attention and Softmax Accelerator for Quantized Transformers,” ISLPED, 2023.
- 9) W. Li et al., “RAWAtten: Reconfigurable Accelerator for Window Attention in Hierarchical Vision Transformers,” DATE, 2023.
- 10) T. Yang et al., “DTQAtten: Leveraging Dynamic Token-based Quantization for Efficient Attention Architecture,” DATE, 2022.
- 11) K. Chen et al., “NOCTUA: A High-Efficiency Reconfigurable NoC-based Transformer Universal Accelerator,” APSIPA ASC, 2025.
- 12) SARCS-IIITH, “Network-on-Chip-NoC,” GitHub repository 